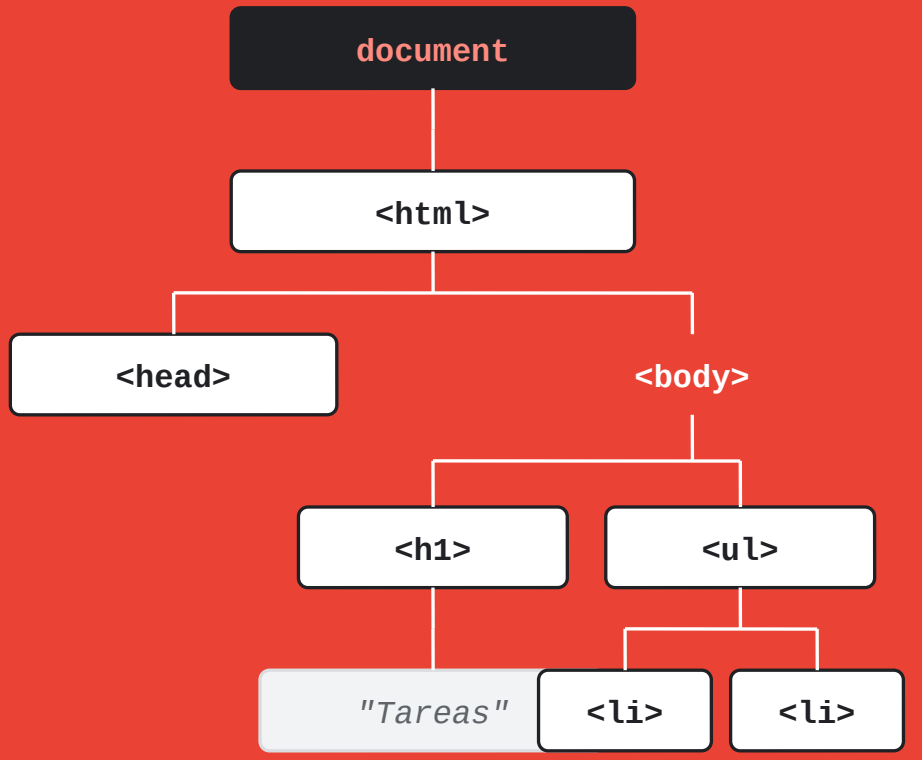
MÓDULO DWEC · DAW — UNIDAD DIDÁCTICA 5

DOM nativo y gestión de eventos

RA4 · Árbol del documento, manipulación dinámica, eventos y formularios

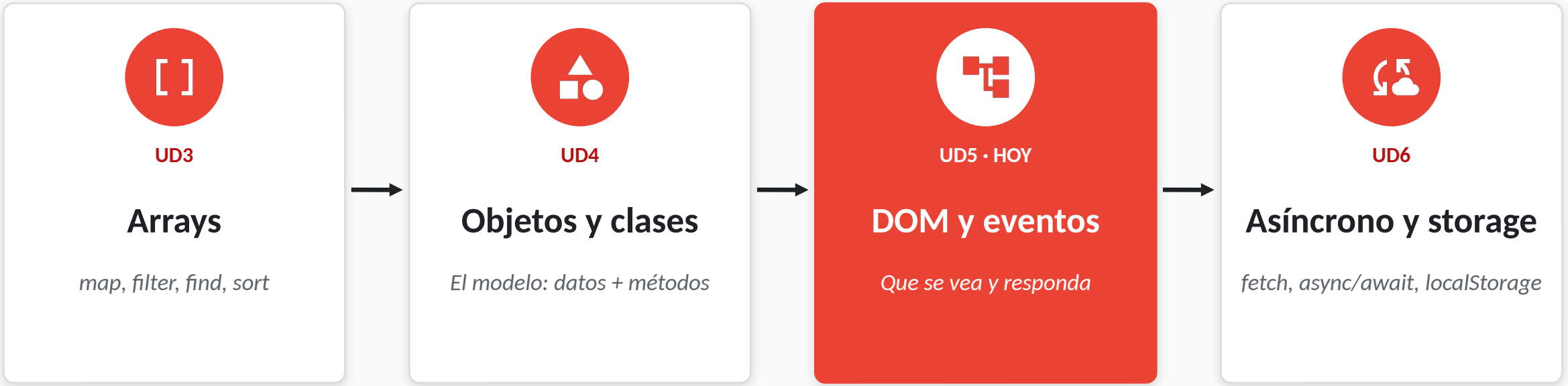
- Entender la página como un árbol de objetos que JavaScript puede cambiar
- Seleccionar, crear y modificar nodos sin recargar y sin abrir agujeros de seguridad
- Responder al usuario con eventos, delegación y formularios validados



Cada caja es un objeto: se puede leer, cambiar, crear o borrar desde JavaScript.

DÓNDE ESTAMOS

Del modelo en memoria a la página que responde



La pregunta de la unidad: ¿cómo hago que lo que pasa en mi modelo se vea en la página, y que lo que hace el usuario en la página cambie el modelo?

OBJETIVO — La Práctica 2 (tablero de tareas) pinta con el DOM el modelo de la UD4: el bloque B es esta unidad aplicada.

CONTEXTO PROFESIONAL

¿Qué pasa cuando pulsas «Añadir al carrito»?

El ciclo de cualquier interfaz



1 · Evento

`el usuario pulsa un botón`

2 · Modelo

`carrito.añadir(producto)`

3 · Render

`se repinta el contador y la lista`

…y vuelta a esperar el siguiente evento.

Sin recargar la página

Gmail, Trello, Spotify web: todo cambia sin pedir una página nueva

Debajo de React y Vue

Los frameworks (UD7) hacen esto por ti, pero con el mismo DOM

Seguridad

Meter datos del usuario con innerHTML es la puerta del XSS

Accesibilidad

Un div con onclick no se usa con teclado; un button, sí

Rendimiento

Tocar el DOM es lo más caro que hace tu JavaScript

DATO CLAVE — El DOM es un estándar del W3C/WHATWG, no de JavaScript: el mismo árbol se maneja desde cualquier lenguaje que hable con el navegador.

HOJA DE RUTA

Cinco bloques, del árbol al formulario

1



El árbol DOM

nodos, document, cuándo está listo

2



Seleccionar y leer

querySelector, textContent, dataset

3



Modificar y crear

classList, createElement, template

4



Eventos

propagación, delegación, this

5



Formularios

FormData, validación, errores

En clase: del árbol al formulario: que la página cambie y responda al usuario. **Para casa:** el laboratorio, el mini-reto y la batería de la UD5, en la web.

CLAVE — Esta unidad se aprende con la consola abierta y DevTools en la pestaña Elements. Lo que no veáis cambiar ahí, no ha pasado.



BLOQUE 1

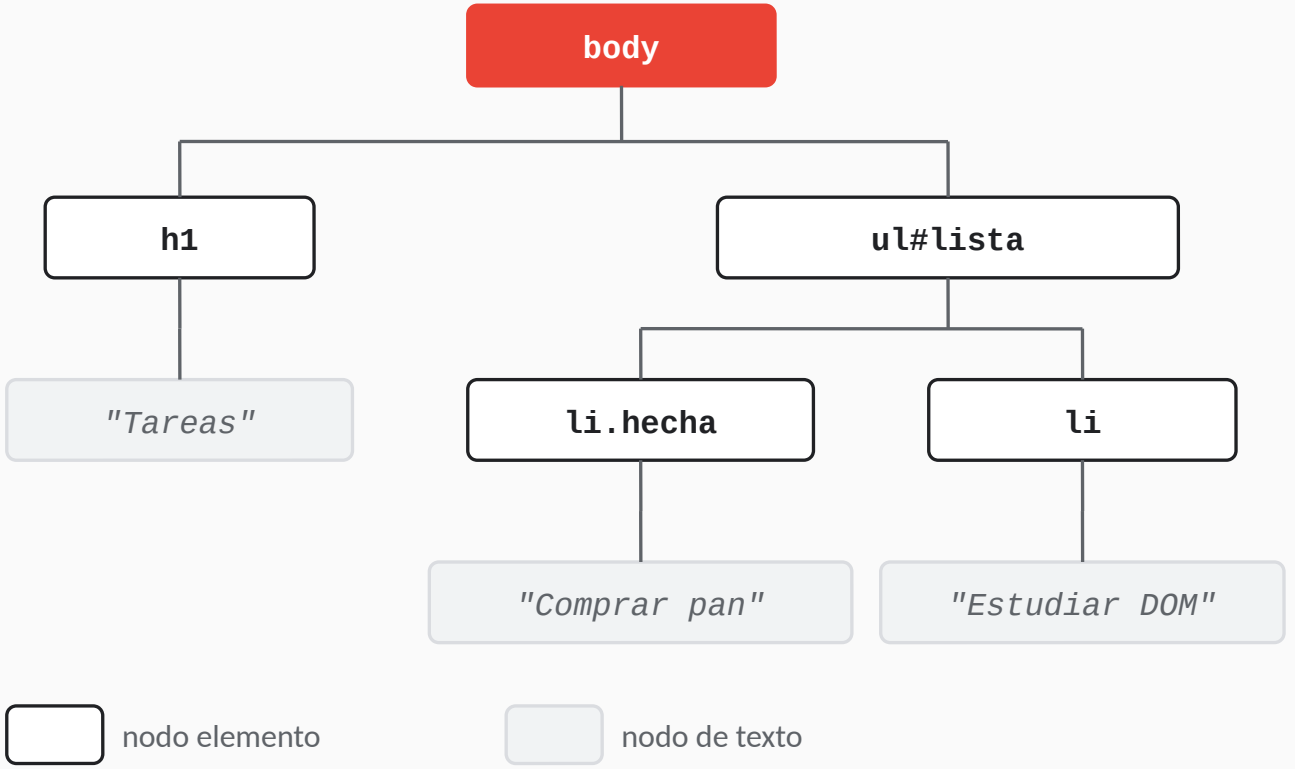
El árbol DOM

Del HTML a un árbol de objetos: nodos, document, window y cuándo está listo

El navegador lee el HTML y construye un árbol de objetos

index.html

```html
<body>
  <h1>Tareas</h1>
  <ul id="lista">
    <li class="hecha">Comprar pan</li>
    <li>Estudiar DOM</li>
  </ul>
</body>
```



Etiquetas → nodos elemento · texto → nodos de texto · anidar → padre e hijos

CLAVE — DOM = Document Object Model: el documento como árbol de OBJETOS. Cambias un objeto y la página cambia al momento.

No todo lo que hay en el árbol es una etiqueta

Tipo	nodeType	Ejemplo	Cómo lo ves
Elemento	1	, <button>	document.querySelector("li")
Texto	3	"Comprar pan", ¡y los saltos de línea!	li.firstChild
Comentario	8	<!-- nota -->	Raro de usar
Documento	9	document	La raíz de todo

consola

```
const ul = document.querySelector("#lista");
ul.childNodes.length;  // 5  ← texto, li, texto, li, texto
ul.children.length;    // 2  ← solo los <li>
ul.firstChild;         // #text "\n    "
ul.firstElementChild;  // <li class="hecha">
```

¿De dónde salen esos nodos de texto?

- Los saltos de línea y espacios entre etiquetas también son texto
- childNodes los incluye; children solo trae elementos
- Usad siempre las versiones Element: children, firstElementChild, nextElementSibling…

NOTA — Node es la clase base; Element hereda de Node, y HTMLLIElement de HTMLElement. Otra cadena de prototipos (UD4).

window es el navegador; document es la página

window (objeto global del navegador · BOM)

document (el DOM)

<html>

<head> · <body>

elementos, textos…

location

history

localStorage

console

setTimeout

navigator

location → URL · history → atrás/adelante · localStorage → UD6

consola

```
window.document === document; // true
window.innerWidth;   // ancho de la ventana
location.href;       // URL actual
document.title = "Hola"; // cambia la pestaña
typeof window;       // "object"
```

Global: en un script clásico, una variable con var o una función declarada acaba en window. En los módulos de Vite, no: cada fichero tiene su propio ámbito.

NOTA — BOM = Browser Object Model: todo lo que el navegador ofrece además de la página. No es un estándar único, pero hoy todos lo implementan igual.

Si buscas un elemento antes de que exista, obtienes null



Cómo cargas el script	¿Cuándo se ejecuta?	¿Ve el body?
<script src> en <head>	En cuanto lo encuentra, parando la lectura	✗ null
<script src> al final de <body>	Al llegar ahí	✓
<script defer src>	Tras leer todo el HTML, antes de DOMContentLoaded	✓
<script type="module" src> (Vite)	Diferido como defer, siempre	✓

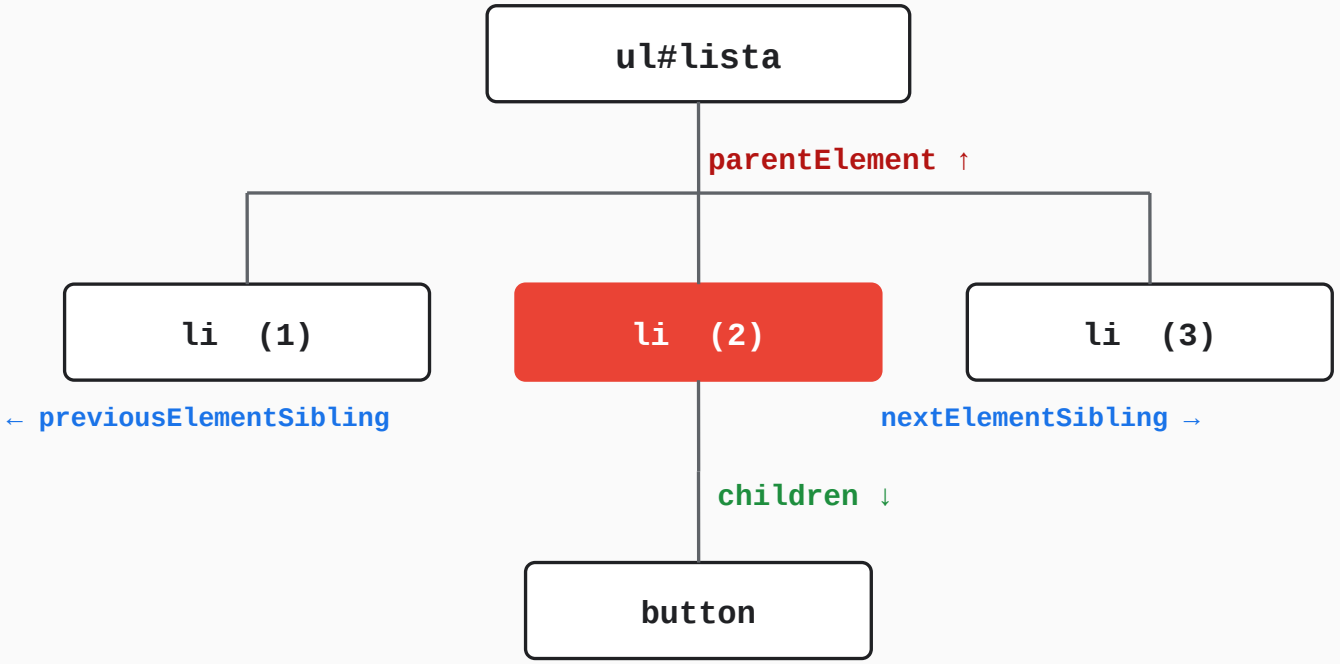
main.js

```
// con type="module" no hace falta,
// pero lo veréis en mucho código:
document.addEventListener(
  "DOMContentLoaded",
  () => iniciar()
);
```

Vite ya pone type="module" en el index.html: el árbol está listo cuando se ejecuta main.js.

CLAVE — «Cannot read properties of null» al empezar el script = buscaste el elemento antes de que existiera (o el selector está mal).

Padres, hijos y hermanos



consola

```
const li = lista.children[1];  // el 2.º
li.parentElement;              // <ul id="lista">
li.previousElementSibling;     // li (1)
li.nextElementSibling;         // li (3)
li.children;                   // [button]
li.firstElementChild;          // <button>

// subir hasta el antecesor que cumpla
const btn = li.querySelector("button");
btn.closest("ul");      // <ul id="lista">
btn.closest(".nada");   // null
```

closest(selector): empieza en el propio elemento y sube por los padres. Es la pieza clave de la delegación de eventos (bloque 4).

NOTA — Moverse por el árbol con nextSibling y compañía es frágil: un cambio en el HTML lo rompe. Preferid seleccionar con querySelector o subir con closest.



BLOQUE 2

Seleccionar y leer

querySelector, colecciones, textContent, atributos frente a propiedades y dataset

BLOQUE 2 · SELECCIONAR

Los mismos selectores que en CSS

Método	Devuelve	Si no hay ninguno
`querySelector(sel)`	El PRIMERO que cumple	null
`querySelectorAll(sel)`	NodeList con todos (estática)	NodeList vacía
`getElementById(id)`	El elemento con ese id (sin #)	null
`el.querySelector(sel)`	Busca solo DENTRO de el	null

Buenas costumbres

- Guarda el elemento en una const al principio: no lo busques en cada clic
- Selecciona por id, clase o data-* propios, no por la estructura (div > div > span)
- Comprueba null si el elemento puede no estar: el?.classList…

seleccionar.js

```
document.querySelector("#lista");
document.querySelector(".hecha");
document.querySelector("ul > li:last-child");
document.querySelector("[data-id='3']");
document.querySelector("input[name=email]");

const items =
  document.querySelectorAll("li");
items.length;
items.forEach(li =>
  console.log(li.textContent));

// dentro de un elemento:
lista.querySelectorAll("li.hecha");
```

CLAVE — Si conocéis los selectores de CSS (DIW), ya sabéis seleccionar en el DOM. Es literalmente la misma sintaxis.

NodeList estática frente a HTMLCollection viva

querySelectorAll → NodeList estática

Foto del momento: si después añades un li, la lista NO cambia. Tiene forEach. No tiene map ni filter.

children / getElementsBy... → HTMLCollection viva

Se actualiza sola al cambiar el DOM. No tiene forEach. Recorrerla mientras borras salta elementos.

estatica-viva.js

```js
const fija = document.querySelectorAll("li");
const viva = ul.children;
ul.append(document.createElement("li"));
fija.length; // 2  (la foto)
viva.length; // 3  (se ha enterado)
```

a-array.js

```js
// ¿quieres map o filter? conviértela
const textos = [...document.querySelectorAll("li")]
  .map(li => li.textContent);
const hechas = Array.from(ul.children)
  .filter(li => li.classList.contains("hecha"));
```

CLAVE — [...lista] o Array.from(lista) y ya tenéis todo lo de la UD3: map, filter, find, some...

textContent, innerText e innerHTML no son lo mismo

tarjeta.html

```
<p id="p">Hola <b>Ana</b>
  <span hidden>oculto</span></p>
```

Para leer y escribir texto, textContent. Es rápido, no interpreta HTML y es seguro (bloque 3). innerText solo si necesitas «lo que se ve».

Propiedad	Devuelve	Ejemplo con #p
`textContent`	Todo el texto, tal cual, también lo oculto	"Hola Ana\n oculto"
`innerText`	El texto como se VE (respeta CSS, más lento)	"Hola Ana"
`innerHTML`	El HTML de dentro, como texto	"Hola Ana\n <span hidden>…"

valores de formulario: otra cosa

```
const email = document.querySelector("#email").value;  // lo escrito AHORA (string)
```

NOTA — Los <input> no tienen texto dentro: lo que el usuario escribe está en .value. textContent de un input es "".

El HTML dice cómo EMPIEZA; la propiedad dice cómo ESTÁ

```html
<input id="cant" value="1">
```

El usuario borra el 1 y escribe 5:

atributo value	propiedad .value
"1" (el HTML original)	"5" (lo que hay ahora)

```
consola
cant.getAttribute("value"); // "1"
cant.value;                 // "5"
check.checked;              // true / false
enlace.getAttribute("href"); // "/ayuda"
enlace.href;  // URL completa: "http://…/ayuda"
```

Para...	Usa
el estado actual de un control	.value · .checked · .disabled
atributos propios o ARIA	setAttribute("aria-expanded", "true")
quitar un atributo	removeAttribute("hidden")
saber si está	hasAttribute("required")
clases y data-*	classList · dataset (siguientes)

Booleanos: hidden, disabled, required, checked valen por existir. setAttribute("disabled", "false") ¡lo DESACTIVA! Usad la propiedad: boton.disabled = false.

CLAVE — Atributo = texto del HTML. Propiedad = valor vivo del objeto. Para controles de formulario, casi siempre la propiedad.

data-* une cada elemento con su dato del modelo

```html
<li data-id="42" data-prioridad="alta" data-fecha-limite="2026-11-06">Entregar</li>
```

`data-id`	→	`dataset.id`	`"42"`
`data-prioridad`	→	`dataset.prioridad`	`"alta"`
`data-fecha-limite`	→	`dataset.fechaLimite`	`"2026-11-06"`

guiones → camelCase · SIEMPRE string

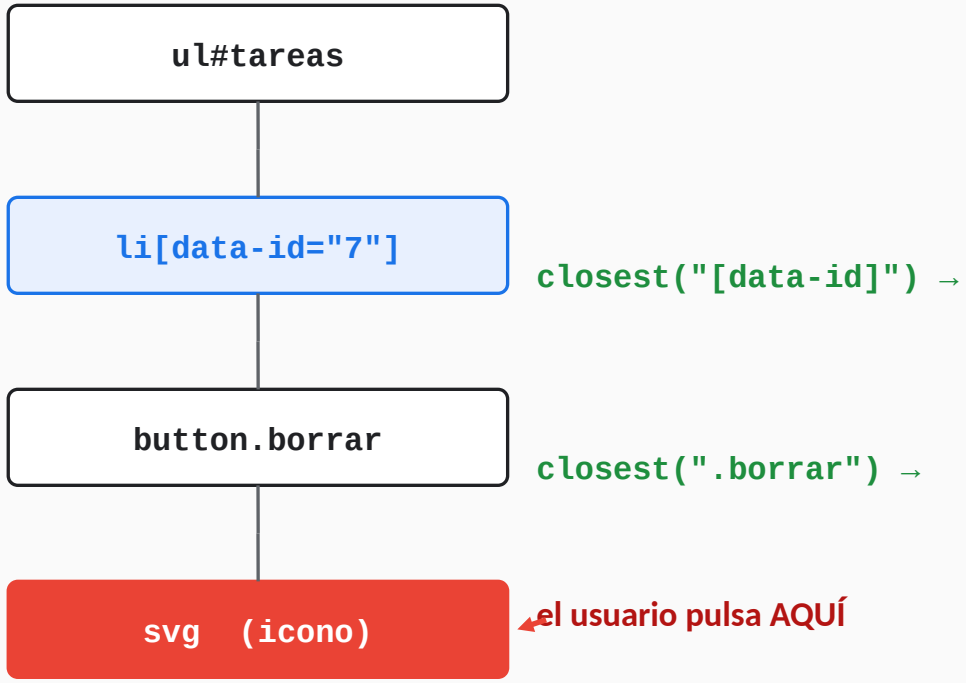
consola

```
li.dataset.id;           // "42"
Number(li.dataset.id);   // 42
li.dataset.hecha = "si";
// → data-hecha="si"
delete li.dataset.hecha;
```

El patrón de toda la unidad: al pintar, guardo el id del objeto en data-id; al hacer clic, leo dataset.id y busco el objeto en el modelo.

CLAVE — Compara con === después de convertir: dataset.id === 42 es false porque "42" es un string. Number(dataset.id) === 42.

¿Este elemento cumple el selector? ¿Cuál de sus padres lo cumple?



consola

```
const el = e.target;          // el svg
el.matches(".borrar");        // false
el.closest(".borrar");        // <button class="borrar">
el.closest("[data-id]");      // <li data-id="7">
el.closest("[data-id]").dataset.id; // "7"

el.closest("form");           // null (no hay form)
```

Por qué importa: el clic puede caer en el icono, en el texto o en el botón. closest encuentra el botón esté donde esté el clic.

NOTA — matches mira solo el propio elemento; closest mira el propio elemento y luego sube. Los dos usan selectores CSS.



BLOQUE 3

Modificar y crear

Texto, clases y estilos, crear e insertar nodos, innerHTML y XSS, template y pintar desde el modelo

El aspecto lo decide el CSS; JavaScript solo cambia la clase

classList.	Hace	Ejemplo
`add("a", "b")`	Añade una o varias	li.classList.add("hecha")
`remove("a")`	Quita	li.classList.remove("hecha")
`toggle("a")`	Pone o quita (devuelve el estado)	li.classList.toggle("hecha")
`toggle("a", cond)`	Fuerza según la condición	li.classList.toggle("hecha", t.hecha)
`contains("a")`	¿La tiene?	li.classList.contains("hecha")
`replace("a", "b")`	Cambia una por otra	btn.classList.replace("off", "on")

toggle con condición es la forma limpia de reflejar el estado del modelo: no hace falta un if/else con add y remove. Y className = "..." BORRA todas las clases que había.

estilos.css

```css
.tarea.hecha {
  text-decoration: line-through;
  opacity: 0.6;
}
.tarea.vencida {
  border-left: 4px solid var(--rojo);
}
```

render.js

```js
li.textContent = tarea.titulo;
li.classList.toggle("hecha",
  tarea.hecha);
li.classList.toggle("vencida",
  tarea.estaVencida());
```

CLAVE — Separar responsabilidades: el CSS sabe cómo se ve «hecha»; el JavaScript solo sabe CUÁNDO una tarea está hecha.

Cuándo sí tocar el estilo directamente

style.js

```
caja.style.backgroundColor = "gold"; // camelCase
caja.style.width = `${porcentaje}%`;  // ¡con unidad!
caja.style.width = 50;                // ignorado
caja.style.removeProperty("width");

// variables CSS (custom properties)
barra.style.setProperty("--progreso", "75%");

// el estilo FINAL, venga de donde venga
getComputedStyle(caja).display;  // "flex"
```

Situación	Mejor
Estados (activo, error, hecha)	classList
Valor calculado (posición, % de progreso)	style o --variable
Temas (claro/oscuro)	clase en <html> + variables CSS
Saber cómo se ve de verdad	getComputedStyle

el.style solo LEE lo que puso style, no lo de las hojas CSS.

Truco limpio: JavaScript pone el número en una variable (--progreso) y el CSS decide qué hacer con ella: width: var(--progreso). Así el diseño sigue en el CSS.

NOTA — style.width = 50 no da error: se ignora en silencio porque falta la unidad. Si algo «no hace nada», mira la pestaña Styles.

BLOQUE 3 · CREAR NODOS

createElement → configurar → append

crear.js

```
function crearItem(tarea) {
  const li = document.createElement("li");
  li.className = "tarea";
  li.dataset.id = tarea.id;

  const texto = document.createElement("span");
  texto.textContent = tarea.titulo;

  const borrar = document.createElement("button");
  borrar.type = "button";
  borrar.textContent = "Borrar";
  borrar.dataset.accion = "borrar";

  li.append(texto, borrar);
  return li;
}
```

1 · createElement — existe en memoria, NO en la página

↓

2 · configurar — textContent, clases, dataset

↓

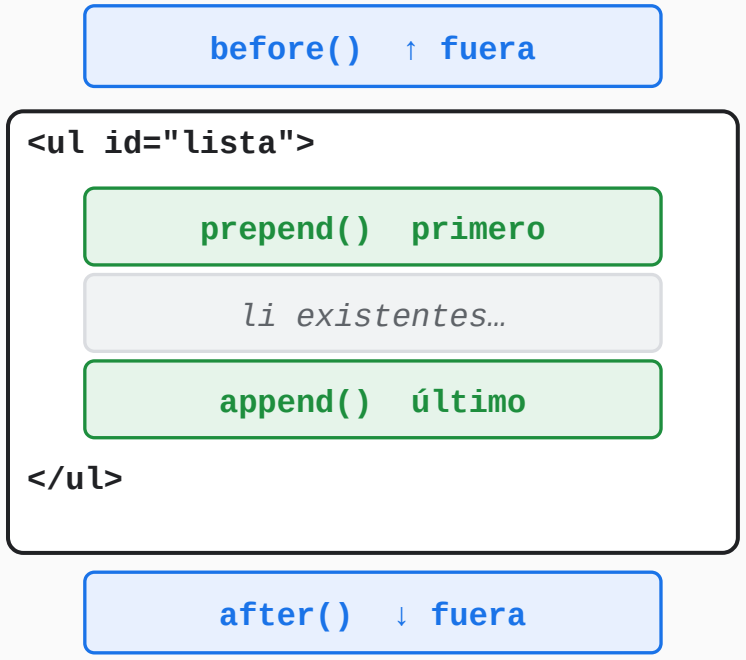
3 · append — ahora sí se ve

```
lista.append(crearItem(tarea));
```

append admite varios nodos y strings; appendChild, uno.

CLAVE — Una función que recibe un objeto del modelo y devuelve un nodo. Esa función es vuestro «componente» antes de React.

Dónde va el nodo nuevo… y qué pasa si ya estaba



dom.js

```
lista.prepend(nuevo);          // primer hijo
lista.append(nuevo);           // último hijo
item.before(nuevo); item.after(nuevo); // hermanos
item.replaceWith(nuevo);       // lo sustituye
item.remove();                 // fuera del árbol
lista.replaceChildren();       // vacía la lista
lista.replaceChildren(...nodos); // vacía y pone estos
```

Un nodo solo está en un sitio: append de un nodo que ya está en la página lo MUEVE, no lo copia. Para duplicar: nodo.cloneNode(true) (true = con sus hijos; los listeners no se copian).

CLAVE — Mover una tarea de «Por hacer» a «En curso» es literalmente columnaEnCurso.append(li). El mismo nodo, otro padre.

Nunca metas datos del usuario con innerHTML

peligro.js

```
const titulo = input.value;  // lo escribe el usuario

// ✗ el navegador INTERPRETA lo que haya escrito
li.innerHTML = `<span>${titulo}</span>`;
```

seguro.js

```
// ✓ textContent no interpreta nada
const span = document.createElement("span");
span.textContent = titulo;
li.append(span);
```

Si escribe: `<img src=x onerror="alert(document.cookie)">`

...su código se ejecuta en la página de todos los que vean esa tarea.

	¿Datos del usuario?
textContent / createElement	✓ Siempre seguro
innerHTML con texto FIJO tuyo	✓ Aceptable
innerHTML con ${datos}	✗ XSS

CLAVE — XSS (Cross-Site Scripting): inyectar código en una web ajena a través de sus propios datos. Sale en todas las ediciones del OWASP Top 10 (desde 2021, dentro de «Injection»).

El HTML de la tarjeta en el HTML, los datos con JavaScript

index.html

```html
<template id="tpl-tarea">
  <li class="tarea">
    <span class="tarea__titulo"></span>
    <time class="tarea__limite"></time>
    <button type="button"
      data-accion="borrar">Borrar</button>
  </li>
</template>
```

render.js

```js
const tpl = document.querySelector("#tpl-tarea");

function crearItem(tarea) {
  const li = tpl.content.firstElementChild
    .cloneNode(true);
  li.dataset.id = tarea.id;
  li.querySelector(".tarea__titulo")
    .textContent = tarea.titulo;
  return li;
}
```

Inerte

no se muestra ni se ejecuta

`cloneNode(true)`

una copia nueva cada vez

`textContent`

rellenar sin riesgo de XSS

NOTA — Con template el maquetador puede cambiar la tarjeta sin tocar JavaScript, y vosotros no escribís createElement veinte veces.

Una sola dirección: el modelo manda y el DOM obedece



main.js

```
function render() {
  lista.replaceChildren(...tablero.tareas.map(crearItem));
  contador.textContent = `${tablero.pendientes} pendientes`;
}
```

La regla: nunca cambies el DOM «a mano» en el manejador. Cambia el modelo y llama a render(). Así lo que se ve siempre coincide con los datos.

CLAVE — Esto es exactamente lo que hace React (UD7): estado → render. Hoy lo hacéis vosotros, y entenderéis qué os ahorra.

Tocar el DOM es caro: agrupa los cambios



lento-rapido.js

```
// ✗ 1000 inserciones = 1000 cambios en la página
for (const t of tareas) lista.append(crearItem(t));

// ✓ un fragmento en memoria y UNA inserción
const frag = document.createDocumentFragment();
for (const t of tareas) frag.append(crearItem(t));
lista.append(frag);
```

Reglas prácticas

- Construye fuera (fragmento o array de nodos) e inserta una vez
- replaceChildren(...nodos) ya inserta todo de golpe
- No leas medidas (offsetHeight, getBoundingClientRect) dentro de un bucle que escribe: fuerza un reflow en cada vuelta
- Para animar, clases con transition/transform (DIW), no style.top en un bucle
- Con 50 tareas no lo notaréis; con 5000, sí

NOTA — El navegador ya agrupa muchos cambios seguidos, pero un DocumentFragment o replaceChildren lo deja claro y evita sorpresas.

ERRORES COMUNES (1/2)

Seleccionar y modificar

El elemento es null

El script corre antes que el HTML, o el selector está mal

```
document.querySelector("lista")
```

```
document.querySelector("#lista")
```

innerHTML con datos

Abre la puerta al XSS

```
li.innerHTML = tarea.titulo
```

```
li.textContent = tarea.titulo
```

map sobre una NodeList

NodeList no tiene map

```
document.querySelectorAll("li").map(…)
```

```
[...document.querySelectorAll("li")].map(…)
```

dataset sin convertir

Siempre es string

```
if (li.dataset.id === 7)
```

```
if (Number(li.dataset.id) === 7)
```



BLOQUE 4

Eventos

addEventListener, el objeto evento, propagación, delegación, listeners duplicados y this

Elemento + tipo de evento + manejador (la función que reacciona)

```
boton .addEventListener( "click" , alPulsar );
```

- `boton` — ¿QUIÉN?
- `"click"` — ¿QUÉ PASA?
- `alPulsar` — ¿QUÉ HAGO? = el MANEJADOR

Manejador *(listener, handler)*: la función que responde al evento. Tú no la llamas: se la das al navegador y él la llama cada vez que pasa.

eventos.js

```js
const boton = document.querySelector("#nueva");

function alPulsar(evento) {   // ← el manejador
  console.log("pulsado", evento.type);
}
boton.addEventListener("click", alPulsar);

// ✗ alPulsar() la EJECUTA ahora y pasa undefined
boton.addEventListener("click", alPulsar());
```

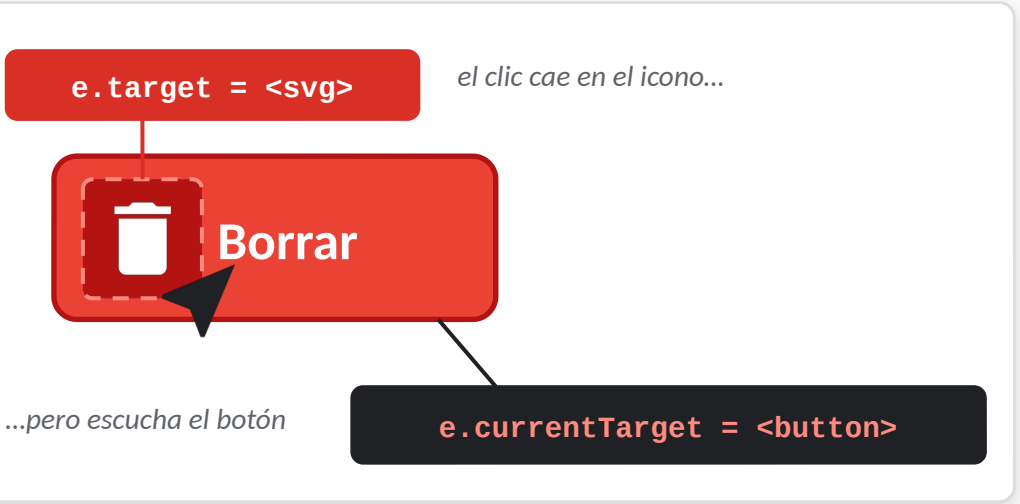


1 · Pulsas el botón → **2 · El navegador** crea el evento → **3 · Llama al manejador** `alPulsar(e)`

Forma	¿Varios?	¿Se usa?
`<button onclick="…">`	✗	No: mezcla HTML y JS
`boton.onclick = f`	✗ el 2.º pisa	Poco
`addEventListener`	✓	Sí, siempre

CLAVE — El manejador es un callback, como los de map o forEach (UD3): le das una función y otro (el navegador) decide cuándo llamarla.

target es dónde ocurrió; currentTarget, quién escucha



Propiedad	Valor en este clic
`e.target`	el <svg> (lo más profundo)
`e.currentTarget`	el <button> (quien escucha)
`e.type`	"click"
`e.timeStamp`	ms desde que cargó la página

evento.js

```
boton.addEventListener("click", (e) => {
  e.target;          // <svg> … o <span>, o el botón
  e.currentTarget;   // siempre el botón
  e.target === e.currentTarget; // depende de dónde
});

campo.addEventListener("keydown", (e) => {
  e.key;     // "Enter", "a", "Escape", "ArrowUp"
  e.shiftKey; // ¿con Mayús?
});

zona.addEventListener("pointermove", (e) => {
  e.clientX; e.clientY; // posición
});
```

CLAVE — Si usas e.target y luego te sale undefined al leer dataset, es que el clic cayó en un hijo. Solución: e.currentTarget o closest().

Los que vais a usar el 90 % de las veces

`click`

Pulsar (ratón, dedo o Enter/espacio en un botón)

→ Botones, enlaces

`input`

Cada cambio en un campo, al momento

→ Buscar mientras escribe

`change`

Al confirmar: soltar el select, marcar, salir

→ select, checkbox, radio

`submit`

Enviar el formulario (botón o Enter)

→ Alta de datos (bloque 5)

`keydown`

Pulsar una tecla (e.key)

→ Atajos, Escape para cerrar

`focusin / focusout`

Entrar o salir de un campo (sí burbujean)

→ Validar al salir

`pointerenter / leave`

El puntero entra o sale (no burbujean)

→ Tooltips

`DOMContentLoaded`

Árbol listo (bloque 1)

→ Arrancar la app

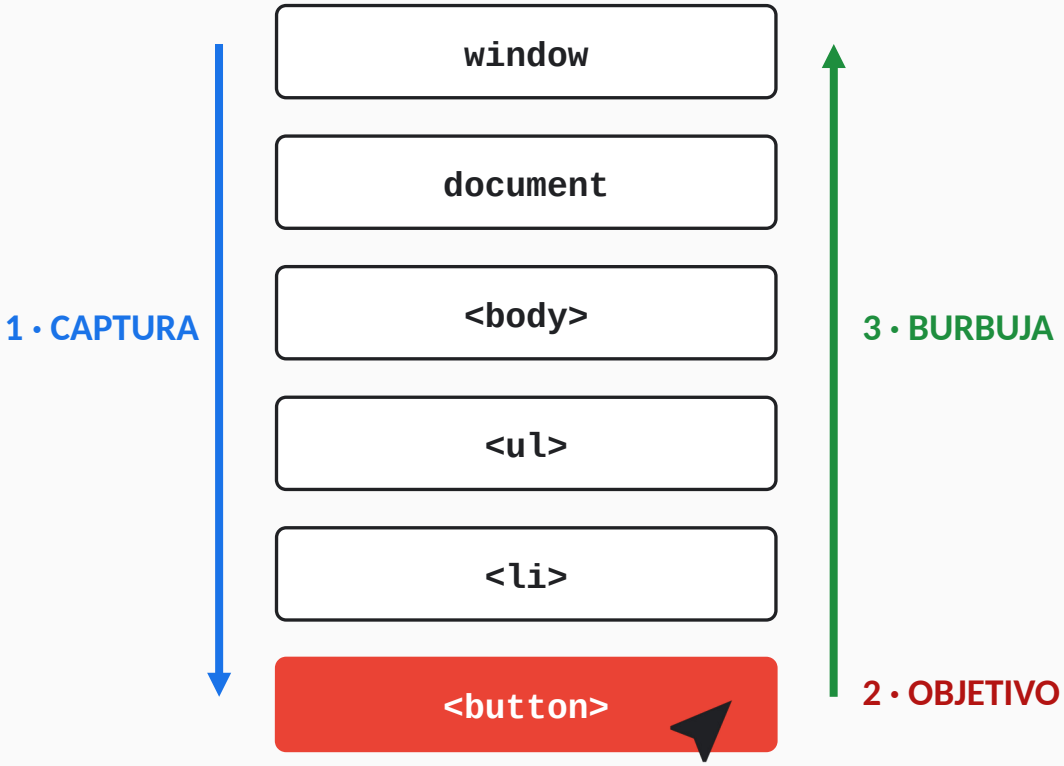
`scroll / resize`

Desplazar o cambiar tamaño: ¡cientos por segundo!

→ Con cuidado

NOTA — click también salta con teclado en <button> y en <a href>. Por eso los botones de verdad son mejores que un con click.

Un clic viaja: baja (captura), llega (objetivo) y sube (burbuja)



consola

```
ul.addEventListener("click",
  () => console.log("ul"));
li.addEventListener("click",
  () => console.log("li"));

// clic en el botón:
// li
// ul
```

Por defecto se escucha en la burbuja.
addEventListener(tipo, f, { capture: true }) escucha al bajar. Casi nunca hace falta.

CLAVE — Gracias a la burbuja, un listener en el <ul> se entera de los clics en TODOS sus <li>. Eso es la delegación (dentro de dos diapositivas).

stopPropagation y preventDefault hacen cosas distintas

e.preventDefault()



Acción del navegador
✗ no ocurre
recargar, seguir el enlace…



Viaje del evento
✓ sigue igual
los padres se enteran

Lo típico: en el submit de un formulario y en enlaces

prevent.js

```
form.addEventListener("submit", (e) => {
  e.preventDefault();   // ¡que no recargue!
  guardar(new FormData(form));
});

enlace.addEventListener("click",
  (e) => e.preventDefault());
```

e.stopPropagation()



Acción del navegador
✓ sigue igual
recargar, seguir el enlace…



Viaje del evento
✗ se para
los padres se enteran

Úsalo poco: rompe la delegación

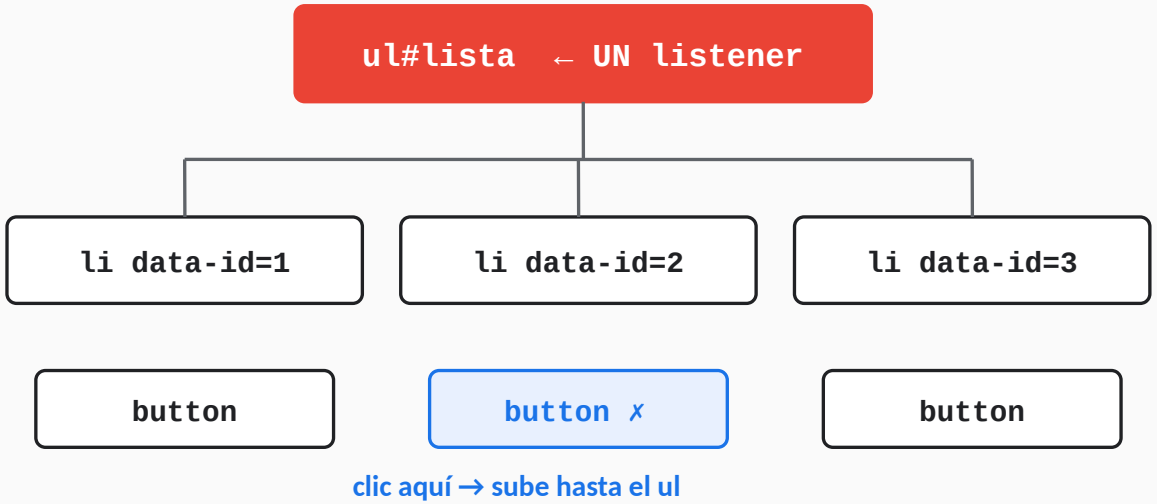
stop.js

```
// modal: clic fuera cierra, clic dentro no
fondo.addEventListener("click", cerrar);
caja.addEventListener("click", (e) => {
  e.stopPropagation(); // no llega al fondo
});
// mejor aún: if (e.target === fondo) cerrar();
```



CLAVE — El fallo n.º 1 de los formularios: olvidar preventDefault en submit. La página se recarga y parece que «no hace nada».

Un solo listener en el padre para todos los hijos, también los futuros



Tres ventajas: un listener en vez de mil, sirve para los elementos que pintes después y no se duplica al repintar.

delegacion.js

```
lista.addEventListener("click", (e) => {
  const boton = e.target.closest("[data-accion]");
  if (!boton) return;          // clic en otro sitio

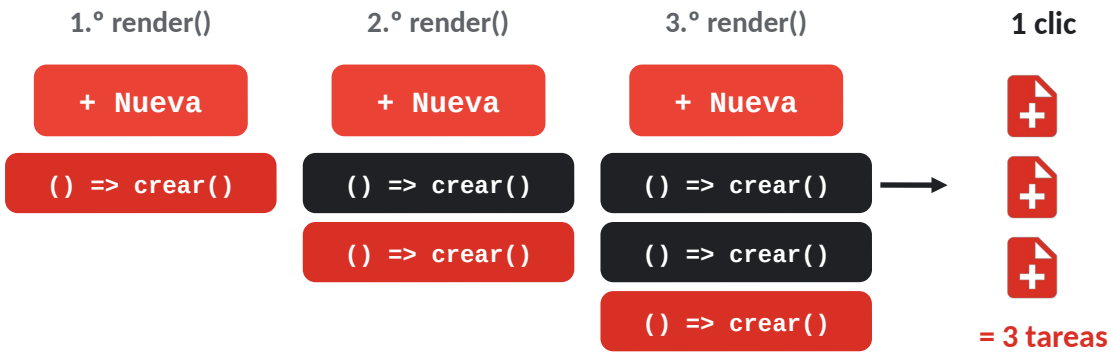
  const id = Number(boton.closest("li").dataset.id);
  const accion = boton.dataset.accion;

  if (accion === "borrar") tablero.borrar(id);
  if (accion === "hecha") tablero.completar(id);
  render();
});
```

CLAVE — El patrón completo: escuchar en el padre → closest para saber qué se pulsó → dataset para saber qué objeto → modelo → render.

Si añades el listener dentro de render(), cada repintado suma otro

```
render() { … nuevaBtn.addEventListener("click", () => crear()); }
```



cada flecha es una función NUEVA: se suman

Arreglos

- Los listeners se ponen UNA vez, al arrancar, fuera de render()
- Para los hijos que se repintan: delegación en el padre
- Los nodos que borras se llevan sus listeners: no hace falta quitarlos

quitar.js

```
// quitar: hace falta LA MISMA función
boton.addEventListener("click", f);
boton.removeEventListener("click", f); // ✓

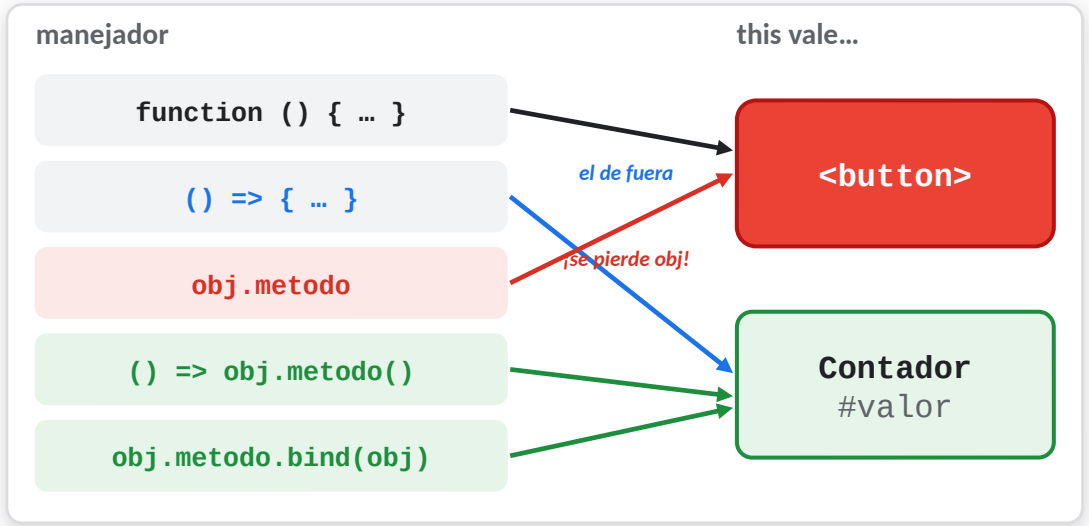
boton.addEventListener("click", () => f());
boton.removeEventListener("click", () => f()); // ✗

// solo una vez
boton.addEventListener("click", f, { once: true });

// quitar varios de golpe
const ctrl = new AbortController();
document.addEventListener("keydown", f,
  { signal: ctrl.signal });
ctrl.abort();
```

NOTA — La misma función añadida dos veces se ignora; una flecha nueva es otra función (=== da false, como dos objetos en la UD4). Por eso se duplica y removeEventListener no la encuentra.

El this perdido de la UD4, otra vez



Con #privados el error se nota: TypeError: Cannot read private member #valor from an object whose class did not declare it. Significa: this no es un Contador.

contador.js

```
class Contador {
  #valor = 0;
  constructor(boton, salida) {
    this.salida = salida;
    // ✗ boton.addEventListener("click", this.sumar);
    //   → this = el botón: this.#valor revienta
    boton.addEventListener("click", () => this.sumar());
  }
  sumar() {
    this.#valor++;
    this.salida.textContent = this.#valor;
  }
}
```

CLAVE — Dentro de clases, la flecha envolvente () => this.metodo() es la solución más legible. Es el mismo arreglo del setInterval de la UD4.

Si se puede pulsar, que sea un <button>

`<div class="btn"> + click`

Tab	Enter	Espacio	
✗	✗	✗	«Borrar» (sin rol)

Hay que añadir a mano tabindex, role y keydown.

`<button type="button"> + click`

Tab	Enter	Espacio	
✓	✓	✓	«Borrar, botón»

Todo gratis. type="button": dentro de un form no lo envía.

teclado.js

```
document.addEventListener("keydown", (e) => {
  if (e.key === "Escape") cerrarModal();
  if (e.key === "n" && e.target.tagName !== "INPUT") {
    nuevaTarea();   // tecla «n» fuera de un campo
  }
});
```

Comprobadlo así

- Desenchufa el ratón: ¿puedes usar toda la app con Tab, Enter y Escape?
- ¿Se ve dónde está el foco? (:focus-visible en el CSS)
- Al abrir algo, mueve el foco con elemento.focus()

NOTA — e.key da la tecla con su significado ("a", "A", "Enter"); e.code, la posición física ("KeyA"). Para atajos, casi siempre e.key.

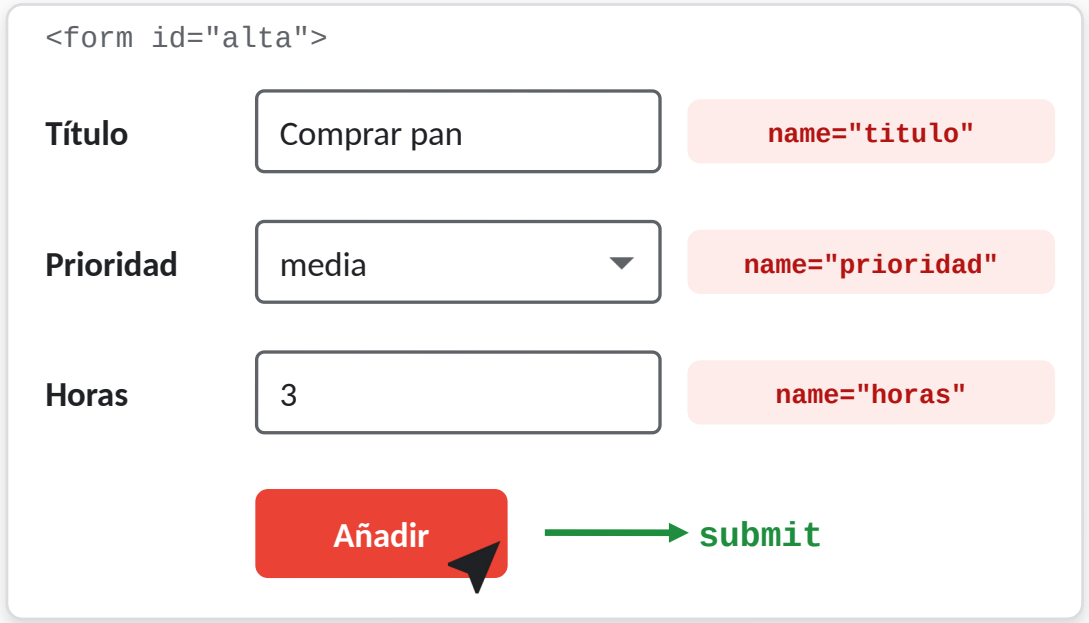


BLOQUE 5

Formularios

Leer datos con FormData, input/change/submit, validación nativa, Constraint Validation y errores accesibles

submit + preventDefault + FormData



alta.js

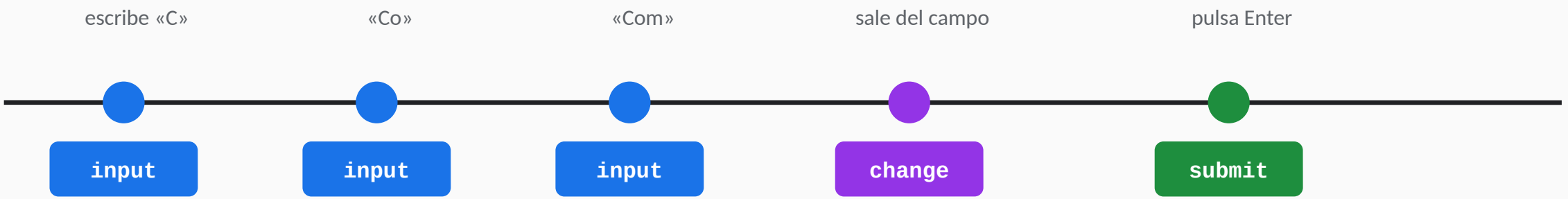
```js
form.addEventListener("submit", (e) => {
  e.preventDefault();
  const datos = Object.fromEntries(new FormData(form));
  // { titulo: "…", prioridad: "media", horas: "3" }
  datos.horas = Number(datos.horas);
  tablero.crearTarea(datos);
  form.reset();
  render();
});
```

`new FormData(form)` → `Object.fromEntries()` → `{ titulo: "Comprar pan", prioridad: "media", horas: "3" }` → `Number("3") → 3`

pares name → valor — *lo convierte en objeto* — *¡todo llega como string!* — *ahora sí, un número*

CLAVE — Escuchad submit en el <form>, no click en el botón: así funciona también al pulsar Enter dentro de un campo.

Cada uno en su momento



buscar.js

```
// filtrar mientras escribe
buscador.addEventListener("input", () => {
  filtro = buscador.value.trim().toLowerCase();
  render();
});

orden.addEventListener("change", render);
```

input	Com|	filtros en vivo, contadores
change	media	select, casillas, validar al salir
submit	Guardar	guardar, SIEMPRE con preventDefault

NOTA — En checkbox y radio, change salta al marcar (no hace falta salir). Lee .checked, no .value.

El HTML ya sabe validar: úsalo primero

Atributo	Comprueba	✗ no pasa	✓ pasa
`required`	que no esté vacío		Ana
`type="email"`	formato de email	ana@	ana@iesx.es
`minlength="3"`	longitud mínima	ab	abc
`type="number" min="1"`	rango (también max, step)	0	2
`pattern="[0-9]{5}"`	regex de TODO el valor	1100	11001

estilos.css

```css
/* solo tras tocar el campo */
input:user-invalid {
  border-color: var(--rojo);
}
input:user-valid {
  border-color: var(--verde);
}
```

novalidate en el <form> desactiva los globos del navegador para mostrar tus mensajes, pero los atributos siguen funcionando para la Constraint Validation API (siguiente diapositiva).

Sin JavaScript: el navegador no deja enviar el formulario si algo falla y muestra su propio mensaje.

NOTA — :invalid marca el campo en rojo desde que carga la página; :user-invalid solo cuando el usuario ya lo ha tocado. Mejor experiencia.

Preguntarle al navegador qué falla en cada campo

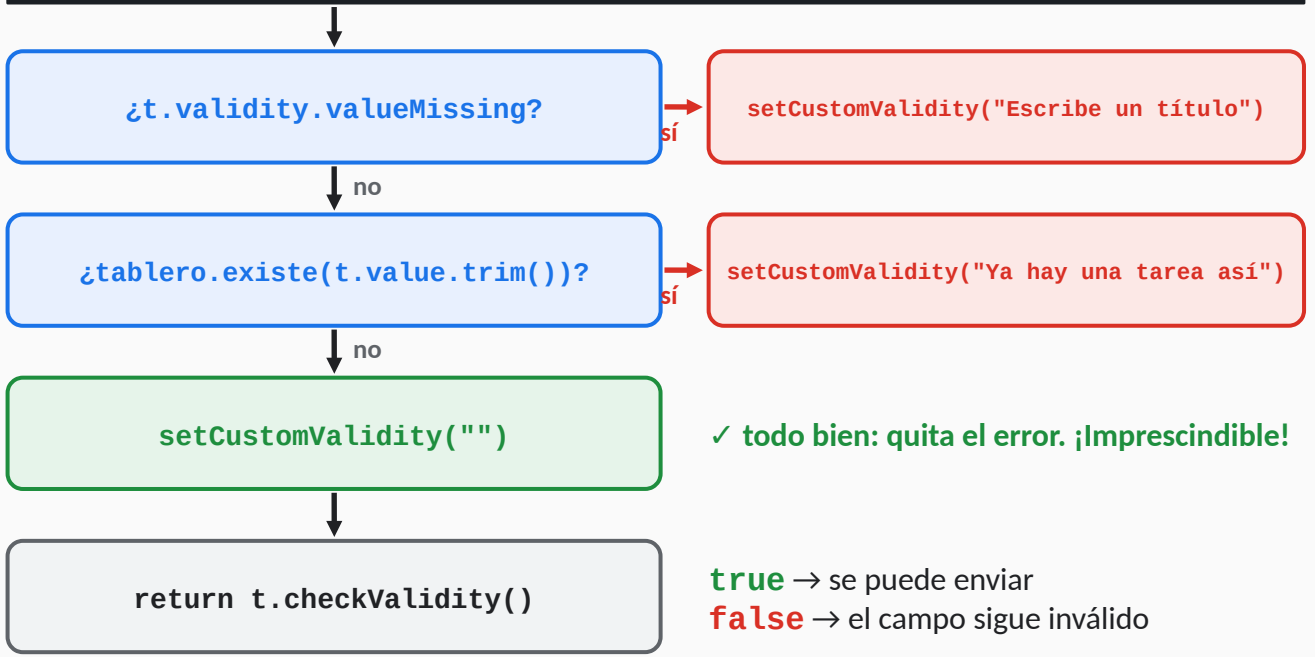
`<input required minlength="3">` el usuario escribe:

ab ✕ **`input.validity` →**

`valueMissing`	`false`	required y vacío
`typeMismatch`	`false`	no parece email / URL
`tooShort / tooLong`	**`true`**	minlength / maxlength (*)
`rangeUnder/Overflow`	`false`	fuera de min / max
`patternMismatch`	`false`	no cumple pattern
`customError`	`false`	setCustomValidity
`valid`	`false`	todo bien

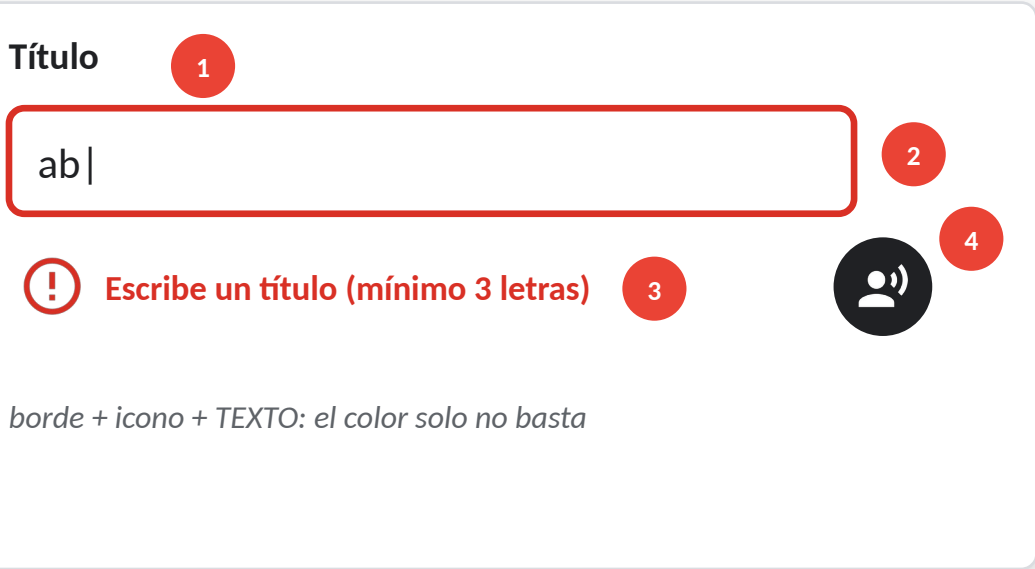
setCustomValidity("") quita el error. Si no la llamas, el campo se queda inválido para siempre.

```
form.elements.titulo.addEventListener("input", validarTitulo);
```



NOTA — (*) Solo con lo que ESCRIBE el usuario: si pones el valor con JavaScript, tooShort sigue en false. form.checkValidity() mira todos los campos.

Junto al campo, en texto, y que el lector de pantalla lo diga



index.html

```html
<label for="titulo">Título</label>
<input id="titulo" name="titulo" required
  minlength="3" aria-describedby="err-titulo">
<p id="err-titulo" class="error" aria-live="polite"></p>
```

1. **`label for + id`** — pulsar «Título» pone el foco en el campo
2. **`aria-invalid="true"`** — el lector avisa de que el campo no es válido
3. **`aria-describedby`** — une el campo con su mensaje de error
4. **`aria-live="polite"`** — el mensaje se lee solo cuando cambia

errores.js

```js
function mostrarError(campo, mensaje) {
  campo.setAttribute("aria-invalid", mensaje ? "true" : "false");
  document.querySelector(`#err-${campo.name}`).textContent = mensaje;
}
```

CLAVE — aria-describedby une el campo con su mensaje; aria-live="polite" hace que se lea al cambiar. Al enviar con errores, pon el foco en el primero.

Es comodidad para el usuario, no una barrera

Usuario normal

ve el error al momento: mejor experiencia

Atacante

abre DevTools, quita required, o manda la petición con curl

Servidor

vuelve a validar TODO: es el único que manda

Regla: valida en el cliente para ayudar, y en el servidor para proteger. En la UD6 enviaréis datos con fetch: el servidor tiene que comprobarlos otra vez. Y nunca confiéis en un precio, un rol o un id que venga del navegador.

NOTA — Lo mismo vale para hidden, disabled o un botón oculto: todo lo que está en el navegador lo puede cambiar el usuario.

GLOSARIO (1/2)

DOM

`DOM`	El documento como árbol de objetos
`nodo`	Cada pieza del árbol: elemento, texto…
`elemento`	Nodo que corresponde a una etiqueta
`document`	Raíz del DOM y punto de entrada
`window / BOM`	El navegador: location, history, storage…
`DOMContentLoaded`	El árbol ya está construido
`querySelector`	Primer elemento que cumple un selector
`NodeList`	Lista de nodos (estática en querySelectorAll)
`HTMLCollection`	Lista de elementos que se actualiza sola
`textContent`	Texto de un nodo; asignarlo no interpreta HTML
`dataset`	Acceso a los atributos data-*
`XSS`	Inyección de código a través de datos

GLOSARIO (2/2)

Eventos y formularios

`evento`	Algo que ocurre: clic, tecla, envío…
`listener`	El manejador: la función que reacciona a un evento
`target`	Elemento donde ocurrió el evento
`currentTarget`	Elemento cuyo listener se está ejecutando
`propagación`	Captura → objetivo → burbuja
`delegación`	Un listener en el padre para todos los hijos
`preventDefault`	Cancela la acción por defecto del navegador
`stopPropagation`	Detiene el viaje del evento
`FormData`	Los campos de un form por su name
`validity`	Estado de validación de un campo
`setCustomValidity`	Mensaje de error propio ("" lo quita)
`aria-live`	Región que el lector anuncia al cambiar

ÚLTIMA REFLEXIÓN

La interfaz es el modelo visto desde fuera

Una sola dirección

Evento → modelo → render. Si el DOM y los datos no coinciden, el fallo está en saltarse ese orden.

Desconfía de la entrada

Todo lo que escribe el usuario es texto: textContent para pintarlo, y el servidor vuelve a validar.

Para todo el mundo

Botones de verdad, foco visible, errores en texto. No es un extra: es parte del trabajo.

Pregunta para casa: abre una web que uses a diario, inspecciona un botón y mira en Event Listeners quién lo escucha. ¿Está en el botón o en un padre?

CHECKPOINT

Resumen de puntos clave

- La página es un árbol de objetos: el DOM
- querySelector + closest + dataset para encontrar elementos y datos
- textContent y createElement; innerHTML nunca con datos del usuario
- Evento → modelo → render(), los listeners una sola vez
- Delegación: un listener en el padre para todos los hijos
- submit + preventDefault + FormData + validity

Hoy: que se vea y responda. Después: que recuerde y hable con el servidor.

SIGUIENTE · UD6

Asincronía y almacenamiento

- Event loop, promesas y async/await
- fetch contra una API
- Guardar el tablero en localStorage